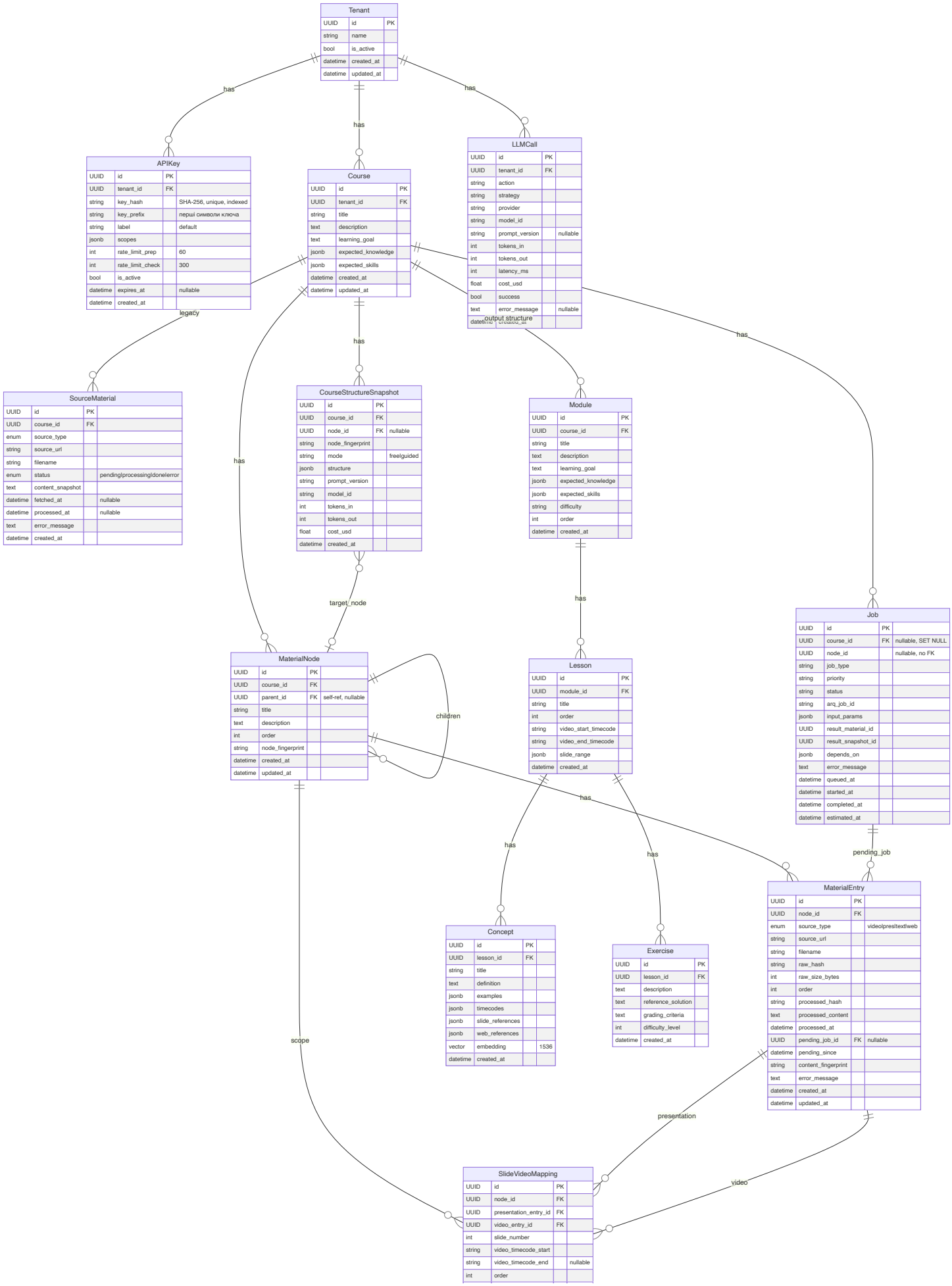
ERD: Course як окрема сутність vs Root Node

Діаграма 1 — Поточна структура (AS-IS)


Tenant
UUID id PK
string name
bool is_active
datetime created_at
datetime updated_at
has
has
has
APIKey
UUID id PK
UUID tenant_id FK
string key_hash SHA-256, unique, indexed
string key_prefix перші символи ключа
string label default
jsonb scopes
int rate_limit_prep 60
int rate_limit_check 300
bool is_active
datetime expires_at nullable
datetime created_at
Course
UUID id PK
UUID tenant_id FK
string title
text description
text learning_goal
jsonb expected_knowledge
jsonb expected_skills
datetime created_at
datetime updated_at
LLMCall
UUID id PK
UUID tenant_id FK
string action
string strategy
string provider
string model_id
string prompt_version nullable
int tokens_in
int tokens_out
int latency_ms
float cost_usd
bool success
text error_message nullable
datetime created_at
legacy
output structure
has
has
has
SourceMaterial
UUID id PK
UUID course_id FK
enum source_type
string source_url
string filename
enum status pending|processing|done|error
text content_snapshot
datetime fetched_at nullable
datetime processed_at nullable
text error_message
datetime created_at
CourseStructureSnapshot
UUID id PK
UUID course_id FK
UUID node_id FK nullable
string node_fingerprint
string mode freeIguided
jsonb structure
string prompt_version
string model_id
int tokens_in
int tokens_out
float cost_usd
datetime created_at
Module
UUID id PK
UUID course_id FK
string title
text description
text learning_goal
jsonb expected_knowledge
jsonb expected_skills
string difficulty
int order
datetime created_at
has
target_node
has
Job
UUID id PK
UUID course_id FK nullable, SET NULL
UUID node_id nullable, no FK
string job_type
string priority
string status
string arq_job_id
jsonb input_params
UUID result_material_id
UUID result_snapshot_id
jsonb depends_on
text error_message
datetime queued_at
datetime started_at
datetime completed_at
datetime estimated_at
MaterialNode
UUID id PK
UUID course_id FK
UUID parent_id FK self-ref, nullable
string title
text description
int order
string node_fingerprint
datetime created_at
datetime updated_at
children
Lesson
UUID id PK
UUID module_id FK
string title
int order
string video_start_timecode
string video_end_timecode
jsonb slide_range
datetime created_at
has
has
has
pending_job
Concept
UUID id PK
UUID lesson_id FK
string title
text definition
jsonb examples
jsonb timecodes
jsonb slide_references
jsonb web_references
vector embedding 1536
datetime created_at
Exercise
UUID id PK
UUID lesson_id FK
text description
text reference_solution
text grading_criteria
int difficulty_level
datetime created_at
MaterialEntry
UUID id PK
UUID node_id FK
enum source_type video|pres|text|web
string source_url
string filename
string raw_hash
int raw_size_bytes
int order
string processed_hash
text processed_content
datetime processed_at
UUID pending_job_id FK nullable
datetime pending_since
string content_fingerprint
text error_message
datetime created_at
datetime updated_at
scope
presentation
video
SlideVideoMapping
UUID id PK
UUID node_id FK
UUID presentation_entry_id FK
UUID video_entry_id FK
int slide_number
string video_timecode_start
string video_timecode_end nullable
int order

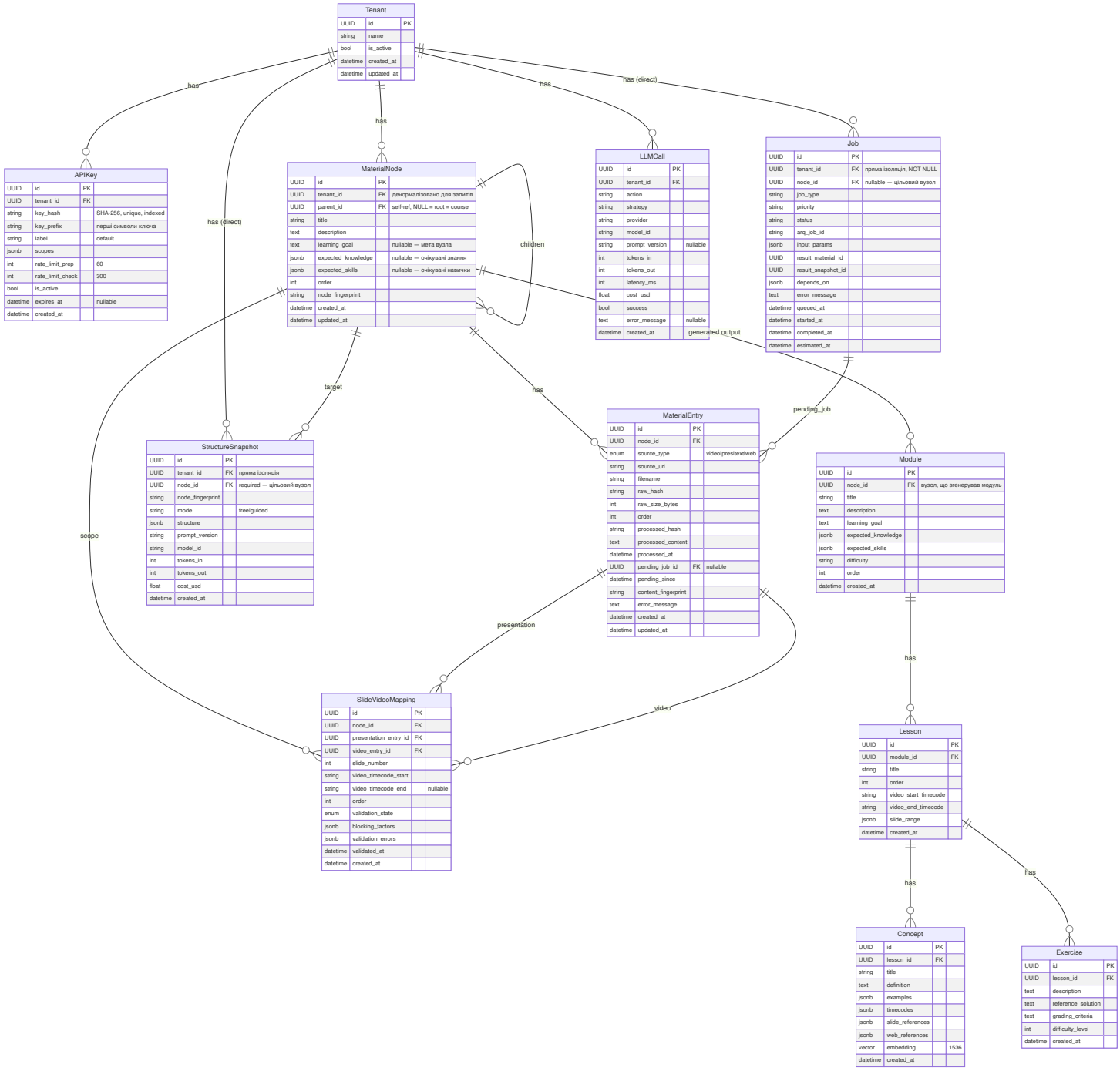
enum	validation_state		
jsonb	blocking_factors		
jsonb	validation_errors		
datetime	validated_at		
datetime	created_at		

Проблеми поточної структури

1. **Дублювання метаданих** — Course і Module мають ідентичні поля (`learning_goal` , `expected_knowledge` , `expected_skills`), а MaterialNode — ту ж ієрархію, але без цих полів.
2. **Course = штучний кореневий рівень** — по суті це root node, винесений в окрему таблицю. Кожен Course має рівно одне дерево MaterialNode. Course не існує без дерева, дерево не існує без Course.
3. **Tenant isolation через JOIN** — Job не має `tenant_id` , ізоляція через `Job → Course → tenant_id` (додатковий JOIN).
4. **SourceMaterial — legacy-дублікат MaterialEntry** — прив'язаний до Course, а не до Node. Має іншу структуру, інший lifecycle.
5. **Рекурсивний аналіз неможливий** — MaterialNode не має полів для learning goals. Коли в майбутньому кожен вузол аналізуватиметься окремо (goal → knowledge → skills → exercises), ці поля потрібні на кожному рівні дерева, а не лише на рівні Course.

Діаграма 2 — Запропонована структура (TO-BE)

Ключова зміна: Course видаляється. Root MaterialNode (parent_id IS NULL) бере на себе роль курсу. Кожен вузол дерева може мати свої learning goals.



Що змінилося

Аспект	AS-IS	TO-BE
Кореневий рівень	Course (окрема таблиця)	MaterialNode з `parent_id IS NULL`
Tenant isolation	Через Course (JOIN)	Пряма: `tenant_id` на MaterialNode, Job, Snapshot
Learning metadata	Тільки Course + Module	Кожен MaterialNode (рекурсивно)
SourceMaterial	Legacy таблиця → Course	**Видалено** (дані мігрують у MaterialEntry)
Snapshot.course_id	Required	Замінено на `node_id` (required)
Job.course_id	FK → Course (SET NULL)	`tenant_id` (direct) + `node_id` (FK)
Module.course_id	FK → Course	`node_id` FK → MaterialNode
Кількість таблиць	14	12 (–Course, –SourceMaterial)

Аспект	AS-IS	TO-BE
JOINs для ізоляції	Job→Course→Tenant	Job→Tenant (прямий)

Що дає нова структура

1. **Рекурсивний аналіз** — кожен вузол має `learning_goal`, `expected_knowledge`, `expected_skills`. LLM може аналізувати будь-який піддерево: від окремого уроку до всього курсу.
2. **Простіша tenant isolation** — `tenant_id` денормалізовано на всіх ключових таблицях. Немає потреби в JOIN через Course.
3. **Менше сутностей** — Course і SourceMaterial видалені. Менше коду, менше repositories, менше міграцій.
4. **Єдина модель матеріалів** — тільки MaterialEntry, без legacy SourceMaterial з іншим lifecycle.
5. **API спрощується** — замість `/courses/{course_id}/nodes/...` можна `/nodes/{root_id}/...` або зберегти backward-compatible alias.

Ризики та відкриті питання

1. **Міграція даних** — на production є курс з 28 матеріалами. Потрібна data migration: Course → root MaterialNode, SourceMaterial → MaterialEntry.
2. **S3 key schema** — поточні ключі `{course_id}/...`. Варіанти: (a) root_node_id = старий course_id (якщо використати той самий UUID), (b) симлінки/аліаси.
3. **API backward compatibility** — 20 endpoints з `course_id` у URL. Варіанти: (a) breaking change (ми ще не маємо зовнішніх клієнтів), (b) alias `/courses/{id}` → `/nodes/{id}`.
4. **`tenant_id` на кожному вузлі** — денормалізація. Потрібен trigger або application-level enforcement щоб child.tenant_id == parent.tenant_id.
5. **Як відрізнити root node від child?** — `parent_id IS NULL`. Потрібен unique constraint: один root per tenant? Чи tenant може мати кілька "курсів" (кілька root nodes)?
6. **Module output прив'язка** — Module.node_id вказує на вузол що згенерував цей модуль. При рекурсивній генерації кожен вузол генерує свої модулі — це правильно?